

Use Case 2: Serverless Architecture

Scenario: Build a serverless photo-upload application.

Tasks:

- o Use S3 for storage.
- o Trigger Lambda functions for image resizing upon upload.
- o Store metadata in DynamoDB.
- o Use API Gateway to expose the functionality.

Step 1: Set Up an S3 Bucket for Storage

Amazon S3 will be used to store the uploaded photos.

1. **Create an S3 Bucket:**
 - o Go to the **S3 console** in AWS.
 - o Click on **Create bucket**.

- Choose a **unique name** for your bucket (e.g., my-photo-upload-bucket).
- Select an appropriate **region**.
- In the **bucket settings**, ensure that versioning and encryption are enabled for better storage management and security.
- You can also enable **public access settings** depending on your needs. If your photos need to be publicly accessible (e.g., for a website), you might want to allow public read access.

2. **Set Up Permissions:**

- Configure the appropriate permissions to ensure only authorized users or services can upload files. Use **IAM roles** with limited permissions to control access.

Step 2: Create a Lambda Function for Image Resizing

AWS Lambda will process the uploaded image and resize it.

1. Create a Lambda Function:

- Go to the **Lambda console** and click on **Create function**.
- Choose **Author from scratch**.
- Give your function a name, such as `resizeImageFunction`.
- Choose an **execution role** for Lambda. This role should have permissions to access S3 (to read and write images) and DynamoDB (for metadata storage).

2. Lambda Function Code:

- The Lambda function will use a package like `sharp` (for Node.js) or Python's **Pillow** to resize the image.

You can use the AWS SDK to interact with S3 and DynamoDB.

Set up Permissions for Lambda:

- Your Lambda function needs permission to access the S3 bucket and read the uploaded image, as well as to write the resized image back to S3. Attach an **IAM role** to the Lambda function with permissions like s3:GetObject and s3:PutObject.

Step 3: Set Up S3 Event Trigger for Lambda

1. **Configure S3 to Trigger Lambda:**
 - Go to your **S3 bucket** where the images are uploaded.
 - Under **Properties**, click on **Events**.
 - Create a new event to trigger the Lambda function when a new file is uploaded.
 - Event type: **PUT** (ObjectCreated).

- Destination: **Lambda function** (select the Lambda function `resizeImageFunction`).

2. **Test the Trigger:**

- Upload a test image to the S3 bucket.
- The Lambda function should automatically be triggered, resize the image, and store it back in S3.

Step 4: Store Metadata in DynamoDB

DynamoDB will be used to store metadata like the file name, user information, image dimensions, and other details.

1. **Create a DynamoDB Table:**

- Go to the **DynamoDB console** and click on **Create table**.
- Set the **Table name** (e.g., `ImageMetadata`).

- Set a **Primary Key** (e.g., imageID as the **partition key**).
 - Optionally, you can add other attributes like userID, originalSize, resizedSize, etc.
2. **Update Lambda to Store Metadata:**
- Modify your Lambda function to store the image metadata in DynamoDB after processing

Step 5: Set Up API Gateway for Exposing Functionality

Amazon API Gateway will be used to expose the functionality as a REST API, allowing external clients to upload images.

1. **Create an API Gateway:**
- Go to the **API Gateway console** and create a new **REST API**.
 - Choose **New API** and give it a name like photo-upload-api.

2. **Create POST Method:**

- Under **Resources**, create a **POST** method that will handle image uploads.
- Choose **Lambda Function** as the integration type, and select the **Lambda function** that handles the image resizing and metadata storage.
- Enable **CORS** if needed for client-side applications.

3. **Deploy API:**

- Deploy the API to a new **stage** (e.g., prod).
- Once deployed, you will receive a **URL endpoint** that can be used to upload photos.

4. **API Call to Upload Image:**

- Clients (e.g., a web or mobile app) can send an HTTP POST request to the API Gateway with the image data. The API

Gateway will forward the request to the Lambda function, which will process the image and store the metadata in DynamoDB.

Step 6: Test the Entire Workflow

1. **Upload Image:**
 - Send a POST request to the API Gateway with the image file.
2. **Check S3:**
 - The uploaded image will be stored in S3.
3. **Lambda Execution:**
 - The Lambda function will be triggered to resize the image, store the resized image in S3, and store the metadata in DynamoDB.
4. **Verify Metadata in DynamoDB:**

- Check DynamoDB to ensure that metadata has been stored correctly.